utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

Here is a comprehensive study guide based on the provided material.

Study Guide: Heaps, Priority Queues, and Java Implementations

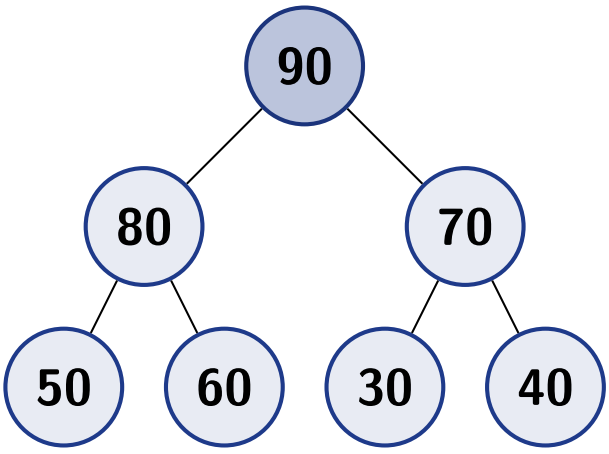
1 Introduction to Heaps

Heaps are essentially tree-based data structures, specifically acting as an additional application of binary trees. A key defining characteristic of heaps is that they are **always complete binary trees**. When discussing heaps, it is necessary to specify if it is a **maximum heap** or a **minimum heap**.

Maximum Heap: The value stored inside any given node is **always bigger** than the values stored inside its children nodes or subtrees. Consequently, the **highest value stored will always be found at the root node**. Unlike binary search trees (BSTs), a heap does not care whether the left child is smaller than the right child; the only requirement is that the **parent is bigger than both of its children**.

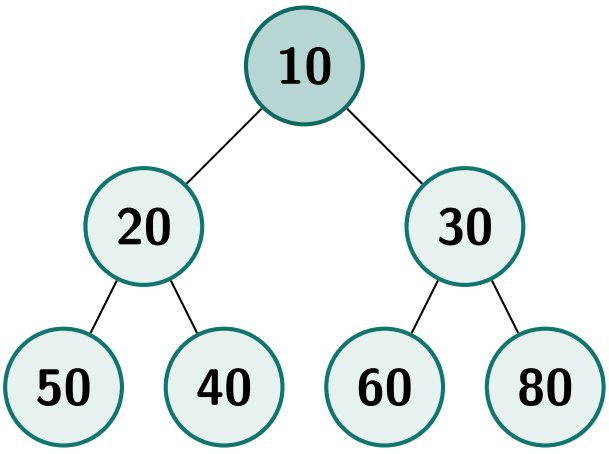
Minimum Heap: The value inside any node is **always smaller** than the values of its children. This ensures the **root node is always the smallest element** in the entire tree.

Maximum Heap Visual Concept



Parent $\geq$ Children (**Root is Maximum = 90**)

Minimum Heap Visual Concept



Parent $\leq$ Children (**Root is Minimum = 10**)

2 Why Use Heaps?

The primary advantage of a heap over a standard binary search tree is **retrieval speed**. Because the maximum or minimum element is **always located at the root**, retrieving it is **extremely**

fast since the root is the first element you have access to. In contrast, finding the minimum or maximum in a BST requires traversing to the far left or far right side of the tree, respectively.

A practical example is storing "customer" objects in a maximum heap. The "biggest" customer can be calculated based on one or many specific fields within the customer class. This means that the maximum or minimum value can be **determined by attributes other than standard mathematical or lexicographic definitions**.

3 Priority Queues

By taking the core concepts of heaps and implementing them, you obtain a highly efficient and powerful implementation of the abstract data type known as a **priority queue**.

- In a priority queue, elements are dequeued based on a specific attribute known as their **"priority"**.
- If a priority queue is implemented using a maximum heap, the element possessing the **highest priority is automatically stored at the root**.
- Because of this structure, the time required to access or dequeue the highest-priority element is optimal, operating in $O(1)$ **time complexity**.

4 Array Representation of Heaps

Even though heaps are theoretically represented as binary trees, they are **primarily implemented in code using arrays** rather than node objects (as you would see in a BST). Because a heap is a complete tree, you can assign each node to an array slot sequentially by **traversing the tree on a line-by-line basis**.

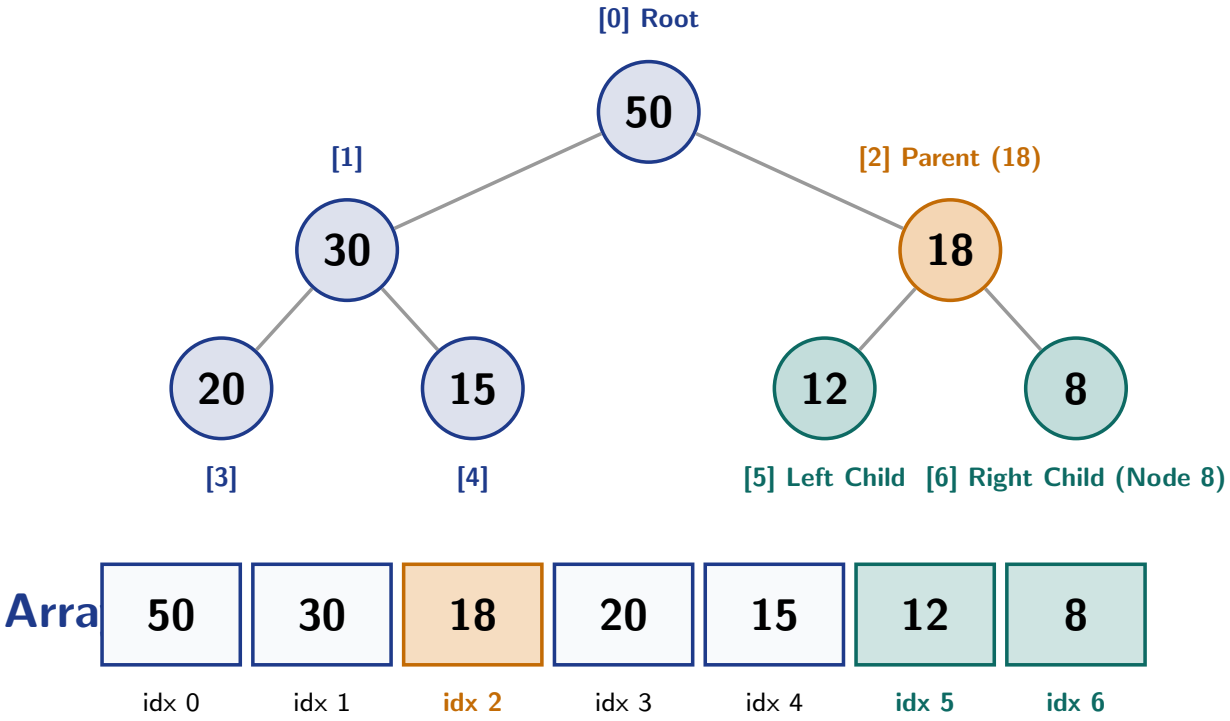
Visualizing Tree-to-Array Mapping:

- The **root node** is positioned at **index 0** of the array.
- The **last node** at the bottom level of the tree sits at the **final index** of the array.
- To locate a specific node's parent or children, specific mathematical formulas connect the array indices.
  - **Left Child: (Parent Index * 2) + 1**. For example, if Node 18 has an index of 2, its left child will have an index of 5.
  - **Right Child: (Parent Index * 2) + 2**. Using the same Node 18 at index 2, its right child will have an index of 6.
  - **Parent: (Child Index - 1) / 2**. If Node 8 has an index of 6, subtracting 1 and dividing by 2 yields 2, which is the index of its parent, Node 18.

5 Core Operations and Java Implementation

5.1 The Heap Interface and Abstract Class

To implement heaps in Java, first create a generic `Heap` interface defining three core methods: `insert` (to add an item), `getRoot` (to remove and return the root), and `sort` (to sort the array



Array Mapping Verification: Left $= (2 \times 2) + 1 = 5$, Right $= (2 \times 2) + 2 = 6$, Parent $= (6 - 1)/2 = 2$

using Heap Sort).

Because Maximum and Minimum heaps share significant logic, the best approach is to create a **generic, abstract `Heap` class** that both specialized classes will extend.

Fields: The class contains a generic array representing the heap and an integer attribute named `position`. The `position` attribute **tracks the last element inside the heap**. If array capacity is 100 but there are only 20 items, `position` equals 19 (since indices start at 0).

Constructor: Initializes the generic array with an **initial capacity of 2**, as it is designed to be a **resizable array**.

5.2 Insertion Operation

When inserting an element, you act as if you are inserting the value at the **very end of the heap array**.

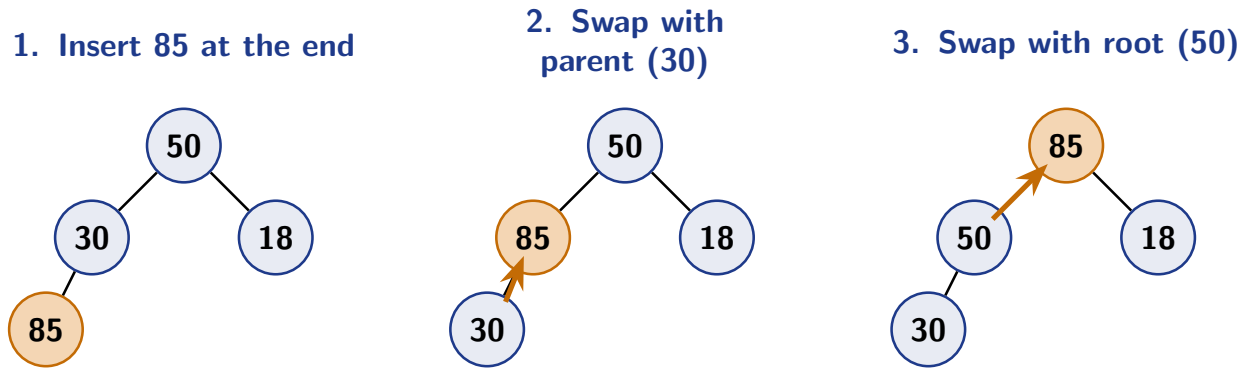
Implementation Steps:

1. **Capacity Check:** Check if the heap is full, which occurs when the `position` integer points to `heap.length - 1`. If full, resize the array by **doubling its capacity** using the `System.arraycopy` method.
2. **Insert Data:** Increment the `position` index (which is initially -1) and assign the user's data to this new position in the heap array.
3. **Fix Upward:** To maintain the heap rule, you must adjust the tree from **bottom to top** using an abstract `fixUpward` method.
4. **Chain Calls:** Return the whole tree at the end of the method to allow for **chaining insertion calls** (optional, but recommended).

fixUpward Implementation:

Maximum Heap: Start at the newly inserted node at the `position` index. Retrieve its index and its parent's index. **While the child's value is greater than the parent's value**, and you haven't reached the top (index 0), **swap the node with its parent**. (The swapping logic itself should live in the abstract class for reuse). After swapping, **hop upwards** by updating your current index variable to be the parent's index, and recalculate the new parent index.

Minimum Heap: The implementation is identical to the Maximum Heap, except the **"greater than" (>)** sign in the `while` loop condition is replaced with a **"smaller than" (<)** sign.



Step-by-Step Visualization of `fixUpward` (Bubbling Up) during Insertion in a Maximum Heap

5.3 Get/Remove Root Operation

The `getRoot` operation **removes and returns the root node**, then restructures the tree to place the new highest (or smallest) value at the root. Unlike insertion, restoring the tree after deletion requires **traversing from top to bottom**.

Implementation Steps:

1. **Empty Check:** Check if the heap is empty. If it is, **return a `null` value** to prevent null pointer exceptions.
2. **Store Root:** If not empty, store the root's value in a temporary result variable so it can be returned later.
3. **Move Last Element:** Take the last element stored in the array (at the `position` index) and **place it at the root (index 0)**. Decrement the `position` attribute to simulate the deletion of this element.
4. **Clean Up:** Because the last element was copied to the root, there are now two identical items. Assign `null` to the last index to remove the redundant item.
5. **Fix Downward:** Traverse the tree downwards, starting at the root and going to the `position` index using an abstract `fixDownward` method.

fixDownward Implementation:

Maximum Heap: Check if the tree is empty. If the tree had only one item before deletion, decrementing `position` makes it -1, requiring no fixing.

If not empty, initialize an index variable to 0.

Run a `while` loop as long as the index is smaller than the provided end index (the `position` index).

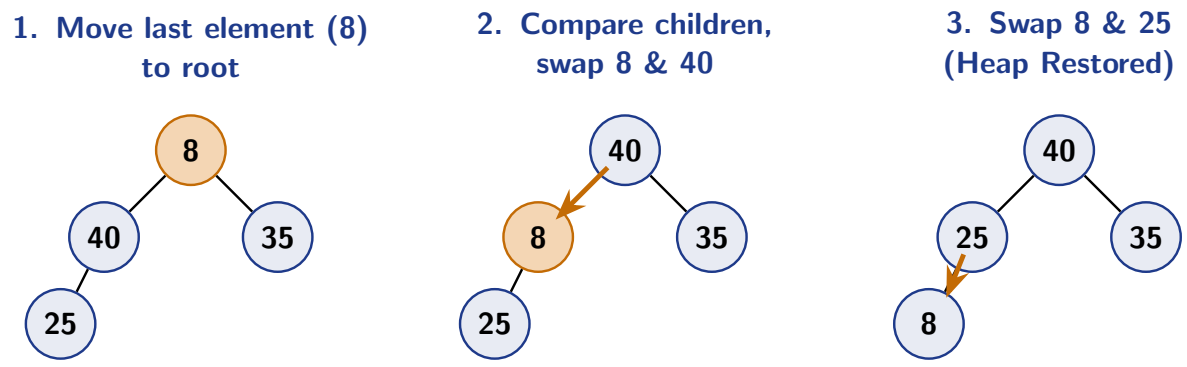
Calculate left and right child indices. Since the right child index is always bigger than the left, first check if the left child index is larger than the end index. If true, **break the loop as the bounds are exceeded** and no swapping is possible.

Determine which child to swap: If the right child index is bigger than the end index, pick the left child. If both are smaller than the end index, **pick the child with the biggest value** using the `compareTo` method.

Check if the chosen child is bigger than the parent. If it is not, the tree is already a valid heap, so exit the loop. If it is bigger, **swap the parent node with the chosen child**.

Hop one node downward by assigning the index variable to the chosen child's index to continue traversal.

Minimum Heap: The logic is exactly the same, but you must replace both "greater than" signs (in the ternary expression determining the child, and in the if-statement comparing child to parent) with a **"smaller than" sign**.



Step-by-Step Visualization of `fixDownward` (Trickle-Down / Heapify) during Root Removal

5.4 Heap Sort Algorithm

Once elements can be inserted and the root retrieved, the elements within the heap array can be sorted using **Heap Sort**.

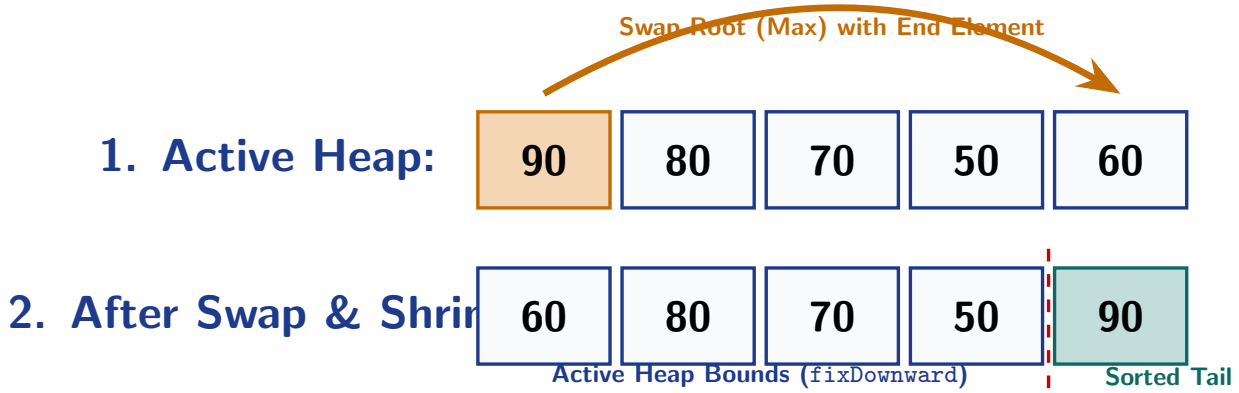
Algorithm Theory: Because the maximum or minimum element is definitively located at the root, you begin by **swapping the root element with the element located at the very end of the array**. Once swapped, consider the new tree to be the entire array minus that final, newly sorted element. Traverse this new tree from top to bottom to find the new maximum (or minimum), place it at the root, and then swap it with the new end element. **Repeat this process until all nodes are sorted**.

Code Implementation:

- Wrap the logic in a `for` loop initialized at index 0, going all the way to the `position` index.
- Inside the loop, use your `swap` method to **swap the root (position 0) with the item at the `position` index minus the current loop index** (to avoid swapping already sorted elements).

- Call the `fixDownward` method to find the new root, providing it with an end index of `position - 1 - current loop index`. This subtracts the number of sorted elements from the bounds so the algorithm doesn't traverse the sorted tail of the array.

Output: If printing the array after calling this method, a maximum heap will print the elements sorted in **ascending order**, whereas a minimum heap will print them in **descending order**.



Heap Sort Step: Swap Max Element to End, Exclude End from Active Range, and Restore Heap via `fixDownward`